

Software Architecture and Design Specification

Field	Detail
Project	Online Internship & Placement Preparation Portal
Version	1.0
Authors	Team 12
Date	16-09-2026
Status	Final

Revision History

Version	Date	Author	Change Summary
1.0	16-09-2026	Team 12	Initial SAD

Approvals

Role	Name	Signature/Date
Project Guide	Ashakumari	_____
Team Representative	_____	_____

1. Introduction

1.1 Purpose

This document specifies the architecture and design of the Online Internship & Placement Preparation Portal. It translates the project SRS into implementable architecture, components, interfaces, security controls and design flows.

1.2 Scope

Covers student and recruiter registration/authentication, internship listing and search, applications and status tracking, timed aptitude tests, student progress dashboards, placement analytics, recruiter verification, administration and notifications.

1.3 Audience

Developers, QA engineers, project guides/instructors, evaluators, administrators and maintenance teams.

1.4 Definitions

JWT – JSON Web Token; RBAC – Role-Based Access Control; REST – Representational State Transfer; OTP – One-Time Password; API – Application Programming Interface; PII – Personally Identifiable Information; TLS – Transport Layer Security; UI – User Interface.

2. Document Overview

2.1 How to use this document

Provides architectural and design deliverables including component architecture, architecture rationale, technology stack, risks, requirement traceability, security architecture, sequence diagrams, API design, error handling, UX design and open issues.

2.2 Related Documents

SRS – Online Internship & Placement Preparation Portal; Software Test Plan (STP); Requirements Traceability Matrix (RTM), where maintained.

3. Architecture

3.1 Goals & Constraints

Goals: secure, modular, responsive and maintainable portal; internship search and applications; timed aptitude preparation and auto-grading; dashboards and analytics; role-based access.

Constraints: MERN-based implementation, MongoDB, HTTPS/TLS, bcrypt, JWT, RBAC, time-bound aptitude-test tokens and the performance, availability and security targets in the SRS.

3.2 Stakeholders & Concerns

Stakeholder	Concerns
Students	Registration, internship search, applications, secure tests, dashboard and progress.
Recruiters	Listing creation, applicant review, status updates and controlled access.
Administrators / Placement Cell	Recruiter verification, management, analytics and auditability.
Developers	Modularity, maintainability, clear APIs and testability.
QA / Evaluators	Traceability, interfaces, error handling, security and test coverage.

3.3 Component (UML) Diagram

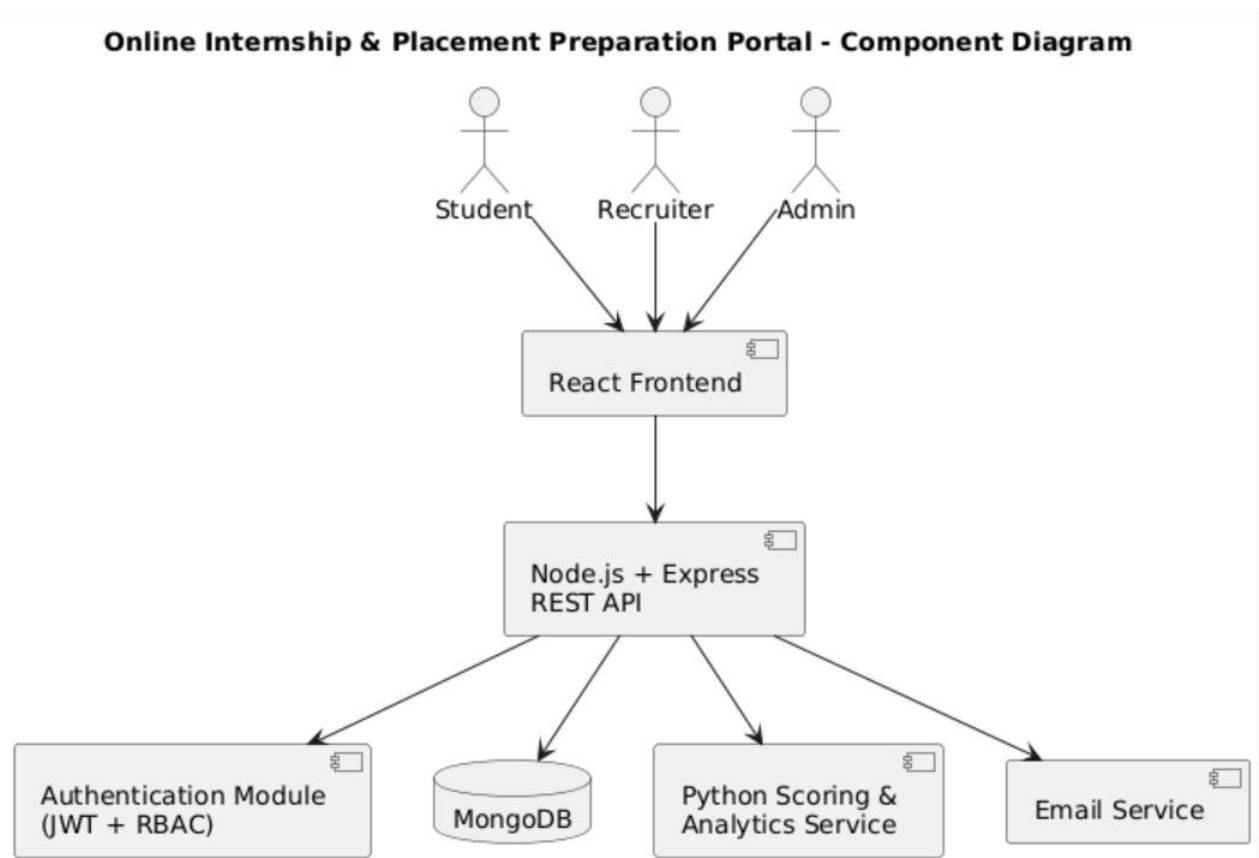


Figure 1. Component UML Diagram – Online Internship & Placement Preparation Portal

3.4 Component Descriptions

Component	Description
React Web UI	Handles user input/output, dashboards, forms, internship search, application screens and aptitude-test interface.
Authentication & Authorization	Registration, verification, login, JWT validation, bcrypt password verification, RBAC and logout.
Node.js + Express API	REST endpoints and application/business logic.
Listing & Application Services	Internship listings, search/filtering, applications and status transitions.
Test & Scoring Services	Timed tests and Python scoring/analytics integration.
MongoDB Atlas	Users, recruiters, internships, applications, tests, attempts, notifications and audit records.
Email Service	Verification, OTP and application/status notifications.
Audit Logging	Authentication and administrative action records.

3.5 Chosen Architecture Pattern and Rationale

Layered architecture is chosen to separate presentation, application/API, data and supporting services. It supports modular implementation, testing and maintenance. Microservices are not selected for the initial project because they add deployment and operational complexity beyond the stated scope.

3.6 Technology Stack & Data Stores

Area	Technology / Design
Frontend	React.js responsive web interface
Backend	Node.js with Express.js REST APIs
Database	MongoDB / MongoDB Atlas with Mongoose
Security	HTTPS/TLS, JWT, bcrypt, RBAC, input validation
Scoring / Analytics	Python scripts/services
Notifications	Email service using SMTP or equivalent provider
Tools	Git/GitHub, draw.io/PlantUML-style UML, API testing/documentation tools

3.7 Risks & Mitigations

Risk	Mitigation
Database/service outage	Managed MongoDB deployment, health checks, monitoring and error handling.
Unauthorized access	JWT, RBAC, ownership checks and recruiter verification.
Credential compromise	bcrypt, TLS and account lockout.
Aptitude-test misuse	Time-bound tokens and server-side session validation.
Email failure	Retry transient failures and record delivery failures.
Slow queries	Indexes, pagination, efficient filters and load testing.
PII/resume exposure	Role-based access, ownership checks and secure file handling.

3.8 Traceability to Requirements

Requirement Group	Architectural Mapping
IPP-F-001 to F-005	Authentication/authorization, React UI, MongoDB and email.

IPP-F-006 to F-009	Listing service, search/filter API, MongoDB and notifications.
IPP-F-010 to F-013	Application service, React UI, MongoDB and notifications.
IPP-F-014 to F-016	Test service, timed tokens, Python scoring and MongoDB.
IPP-F-017 to F-019	Analytics/scoring, dashboard UI and MongoDB.
IPP-F-020 to F-021	Admin UI/API, RBAC and MongoDB.

3.9 Security Architecture

Security uses TLS 1.2+, salted bcrypt password hashing, JWT authentication, RBAC, server-side logout, recruiter verification, time-bound aptitude-test tokens and audit logging. Threat considerations include spoofing → authentication; tampering → HTTPS and validation; information disclosure → RBAC and ownership checks; denial of service → rate limiting; elevation of privilege → RBAC and admin-only operations.

4. Design

4.1 Design Overview

The portal uses layered modules for separation of concerns. React manages presentation; Express handles business logic and security middleware; MongoDB provides persistence; Python and email services provide scoring, analytics and notifications.

4.2 UML Sequence Diagrams

The following project-specific sequence flows illustrate the interaction between the Student, React Frontend, Node.js/Express API, database, authentication module, and scoring service: (1) Student Login, Internship Search and Application; and (2) Timed Aptitude Test and Automatic Scoring.

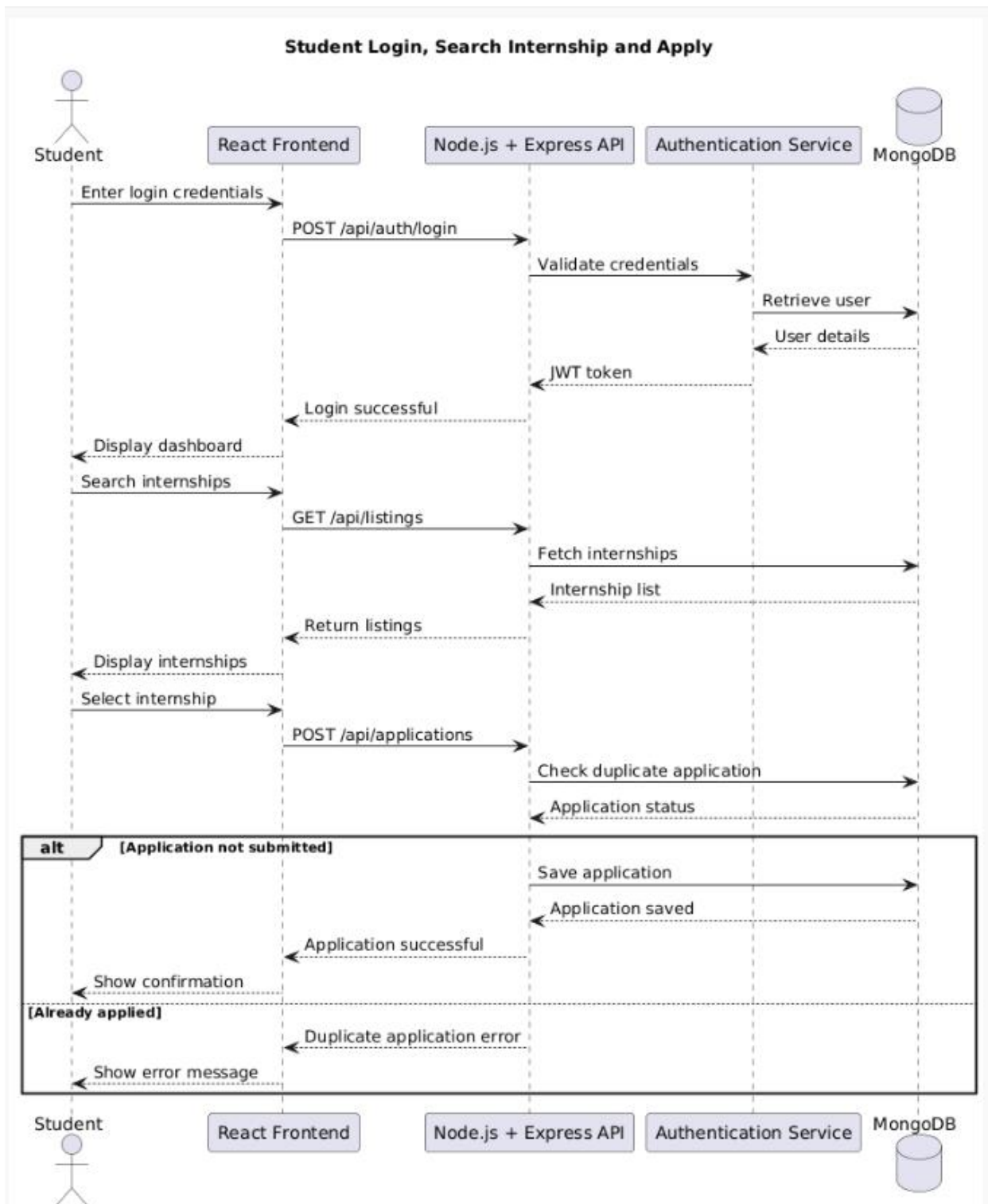


Figure 2. Sequence Diagram – Student Login, Search Internship and Apply

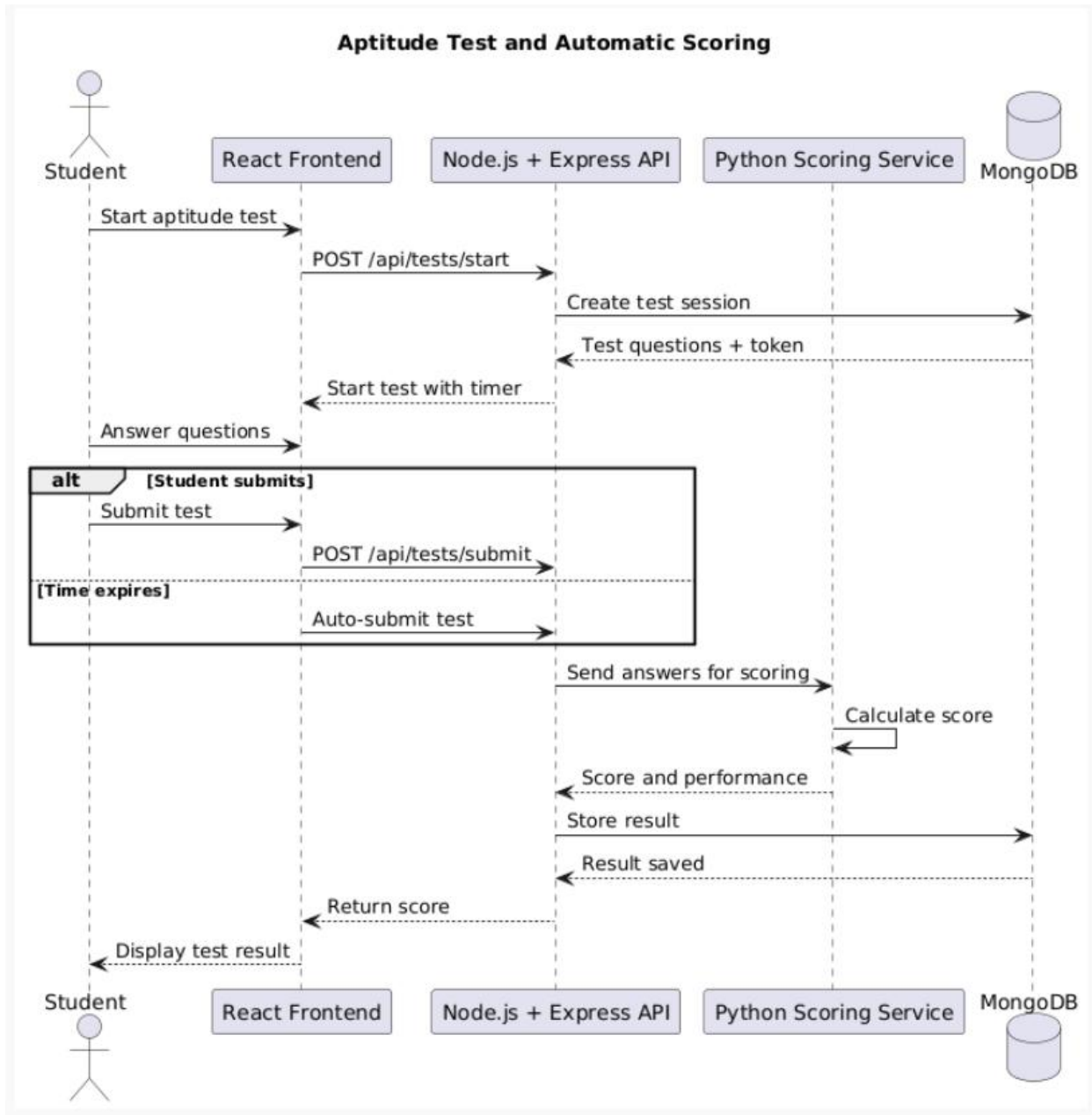


Figure 3. Sequence Diagram – Aptitude Test and Automatic Scoring

4.3 API Design

Method	Endpoint	Request / Input	Response / Output	Errors
POST	/api/auth/login	{email,password}	{token,user}	401 invalid; 429 rate limit
GET	/api/listings	Query filters	{listings,pagination}	400 invalid; 500 server

				error
POST	/api/applications	{listingId,resumeRef,coverLetter}	{applicationId,status}	400 invalid; 409 duplicate
POST	/api/tests/start	{testId}	{sessionToken,questions,expiresAt}	401/403 unauthorized; 404 not found
POST	/api/tests/submit	{sessionToken,answers}	{score,attemptId}	400 invalid; 401/403 expired
PATCH	/api/applications/:id/status	{status}	{applicationId,status}	401/403 unauthorized; 404 not found

4.4 Error Handling, Logging & Monitoring

Use standardized HTTP status codes and safe client-facing messages. Do not log passwords, OTPs or sensitive internal errors. Monitor API response time, failed authentication rate, application failures, test submission failures, database errors and service availability.

4.5 UX Design

Separate Student, Recruiter and Admin dashboards; clear validation messages; internship search filters; visible aptitude-test countdown and submission status; responsive desktop/mobile UI; accessibility aligned with the project requirement.

4.6 Open Issues & Next Steps

Open Issue / Next Step	Planned Action
Finalize production email provider	Configure provider and integration testing.
Finalize deployment	Configure staging/production with HTTPS.
Finalize analytics boundary	Confirm Python analytics interface and dashboard metrics.
Update UML/RTM	Revise when implementation requirements change.
Security validation	Perform security-focused integration testing before release.

5. Appendices

5.1 Glossary

Term	Meaning
API	Application Programming Interface
JWT	JSON Web Token
RBAC	Role-Based Access Control
OTP	One-Time Password
PII	Personally Identifiable Information
REST	Representational State Transfer
TLS	Transport Layer Security
UI	User Interface

5.2 References

Software Requirements Specification – Online Internship & Placement Preparation Portal, Version 1.0. Software Test Plan – Online Internship & Placement Preparation Portal, Version 1.0. OWASP Top 10 (2021). WCAG 2.1 AA.

5.3 Tools

Git and GitHub; draw.io / PlantUML-style UML diagramming; React.js; Node.js/Express; MongoDB/Mongoose; Python; API testing/documentation tools.